From a physics model to a computational experiment

Week 1 | Programming fundamentals through radioactive decay

**A sample has a half-life of 6.0 h.
How should we simulate one day — and decide whether the answer deserves trust?**

LECTURE SESSION

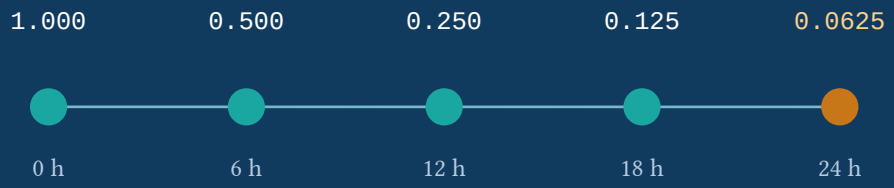
model → code → evidence

THE WORKED EXPERIMENT

Radioactive decay

$$N(t) = N_0 \exp(-\lambda t)$$

T_half = 6.0 h **tMax = 24.0 h**



The number at 24 h is a reference, not yet a numerical method.

Today's arc: ask → compute → test

A laboratory-style computational physics session

1. **Physical question**
2. **MATLAB essentials**
3. **Decay model**
4. **Euler experiment**
5. **Validation + Lab bridge**

Learning outcomes

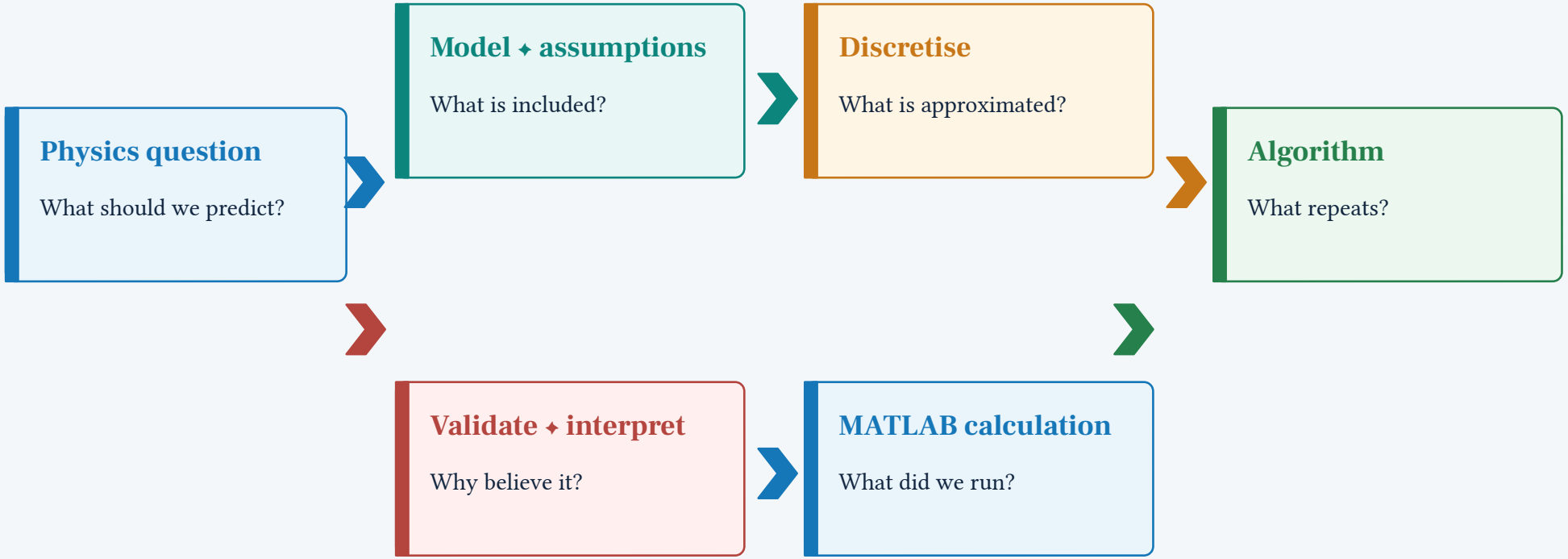
- ★ Map physical quantities to variables and units
- ★ Implement one readable time-step loop
- ★ Compare with a reference and justify dt
- ★ Record evidence so another person can rerun it

★ **Course roadmap:** Week 1 foundations → later numerical methods

CHECKPOINT / 12–18 min

A computational experiment is a loop, not a single command

Each arrow carries a decision that should remain visible.



The last question loops back to the first: does the computation answer the physics question?

Rule of thumb
A graph is evidence only when the model, units, algorithm, and checks are visible.

Four views, one repeatable computational experiment

Know where to try, save, inspect, and run.

Command Window

Try one expression and inspect its answer.

```
>> log(2)
ans = 0.6931
```

Editor / Live Editor

Save the sequence of choices: text + code + output.

This becomes the experiment record.

Workspace

Variables currently in memory.

Useful for inspection — unsafe as a hidden input.
Start fresh with clearvars.

Current Folder

The files MATLAB can see.

Keep the Live Script and related files together.

FRESH-RUN HABIT

```
clearvars; close all; clc
```

Readable names make units and assumptions harder to hide

The syntax is small; the naming discipline is the real habit.

PARAMETER RECORD

```
% name the quantity and the unit
radius_m = 0.02;          % m
area_m2 = pi*radius_m^2;  % m^2

T_half_h = 6.0;
lambda_per_h = log(2)/T_half_h; % 1/h
```

Four ideas to notice

1. **Assignment**
   Use = to store a value in a named variable.

2. **Operators**
   Use +, -, *, /, and ^ to translate an expression.

3. **Comments**
   Use % to explain a choice to the next reader.

4. **Unit naming**
   A suffix such as _h or _m2 keeps dimensions visible.

Quick check

What are the units of lambda_per_h * dt_h?

A unit conversion is already a small computational model

Make the conversion path explicit before asking MATLAB for a number.

Given

Tank volume

2 gallons + 4 pints

8 pints = 1 gallon
1.76 pints = 1 litre

Question
What is the volume in litres?

TRANSPARENT TRANSLATION

```
gallons = 2;
pints = 4;
pintsPerGallon = 8;
pintsPerLitre = 1.76;

totalPints = gallons*pintsPerGallon + pints;
volumeLitres = totalPints/pintsPerLitre;

fprintf('Volume = %.3f L\n', volumeLitres)
```

EXPECTED OUTPUT / 11.364 L

Lab 01 also uses faulty statements and a quadratic-formula warm-up — the same syntax habits apply.

Radioactive decay gives us a controlled question

A model is an equation plus the assumptions that make it useful.

Question

A closed sample has a constant half-life of 6.0 h.

How many undecayed nuclei remain after one day?

MODEL EQUATION

$$\frac{dN}{dt} = -\lambda N$$

$$N(0) = N_0$$

Included

Constant lambda
Closed sample
Same time unit

Ignored

Changing decay
Inflow or outflow
Interactions

Predict

At 6 h?
At 24 h?
Can $N < 0$?

Map physics symbols to MATLAB names before coding

Use names that expose meaning and time units.

Physics	MATLAB name	Meaning / unit
N0	`N0`	initial population / nuclei
T_half	`T_half_h`	half-life / h
lambda	`lambda_per_h`	decay constant / 1/h
tMax	`tMax_h`	final time / h
dt	`dt_h`	numerical step / h

PARAMETER RECORD

```
N0 = 1000;
T_half_h = 6.0;
tMax_h = 24.0;
dt_h = 0.5;
lambda_per_h = log(2)/T_half_h;
```

Dimension check

```
lambda_per_h * dt_h

(1/h) * h = 1

The factor in the Euler update is
dimensionless.
```

The exact answer is a benchmark for the numerical experiment

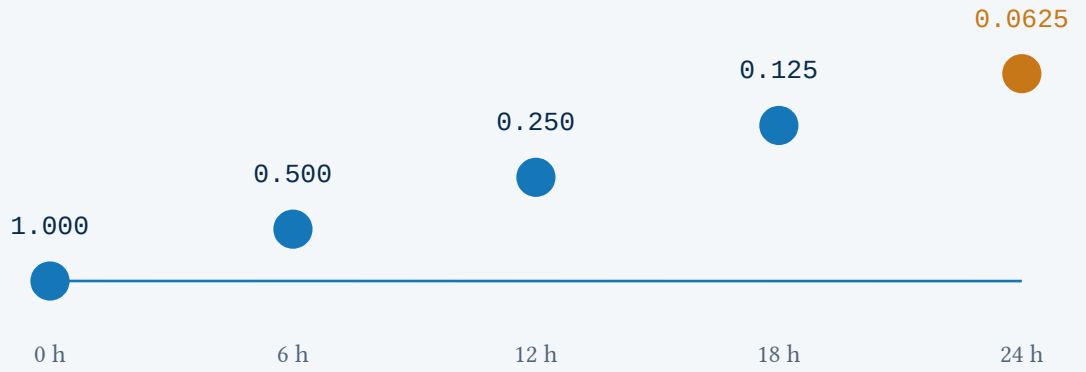
It tells us what to expect before we write a loop.

EXACT SOLUTION

$$N_{exact}(t) = N_0 \exp(-\lambda t)$$

$$\lambda = \ln(2) / T_{half} = 0.1155 \text{ 1/h}$$

At tMax = 24 h: N/N0 = 0.0625



Prediction before compute

1. **One half-life**
   The exact fraction should be 0.5.
2. **Four half-lives**
   The exact fraction should be 0.0625.
3. **Physical sign**
   N < 0 would be an implementation warning.

The reference is not the experiment

We still need to carry out the discrete update, compare it with this benchmark, and test the timestep. The point is to make approximation visible.

Forward Euler turns a derivative into a next-value rule

The computer repeats one local approximation across the time array.

Start with the model

```
dN/dt = -lambda N
```

Approximate the slope

```
(N(n+1) - N(n)) / dt  ≈
-lambda*N(n)
```

Rearrange

```
N(n+1) = N(n)*(1 -
lambda*dt)
```

What changed? The physics model stayed the same. Only time was discretised, so dt becomes part of the numerical method and must be tested.

Smaller dt

Usually improves the approximation here.

More steps

Increases the computational cost.

Large lambda*dt

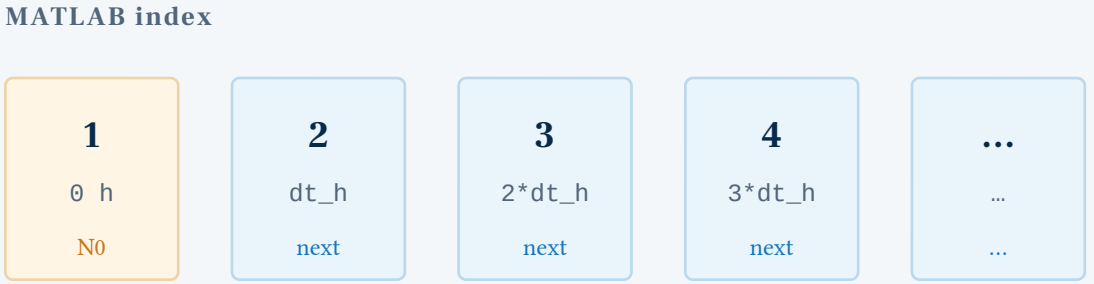
Can make N negative — unphysical.

Build the aligned time array before the loop

The first element stores the initial condition; the rest are destinations.

STORAGE PLAN

```
t_h = 0:dt_h:tMax_h;
N = zeros(size(t_h));
N(1) = N0;
```



One time value ↔ one population value

Preallocation

zeros(size(t_h)) makes the storage plan explicit before values are filled.

Initial condition

N(1) = N0 is not optional. It anchors the entire history.

Alignment check

The arrays must have equal length before plot(t_h, N/N0).

The for loop should look like the algorithm

Read current value → apply rule → store next value.

THE REPEATED UPDATE

```
for n = 1:numel(t_h)-1
    N(n+1) = N(n)*(1 - ...
        lambda_per_h*dt_h);
end
```

1 **Read current value**
N(n) belongs to the current time t_h(n).

2 **Apply the rule**
Multiply by the dimensionless factor 1 - lambda*dt.

3 **Store next value**
N(n+1) belongs to t_h(n+1), preserving the history.

Classic off-by-one error

Writing N(n) on the left overwrites the current value.

Hold the physics fixed; vary one numerical choice

A fair experiment changes dt and records both result and cost.

What stays fixed?

Physical model

```
N0 = 1000
T_half_h = 6.0 h
tMax_h = 24.0 h
lambda_per_h =
log(2)/T_half_h
```

What changes?

Numerical resolution

```
dtValues_h = [2.0 1.0 0.5
0.1]

Same update rule
Different number of steps
```

What do we record?

Evidence

N_Euler(24 h)/N0
relative error
step count
plot against exact reference

EXPERIMENT SKELETON

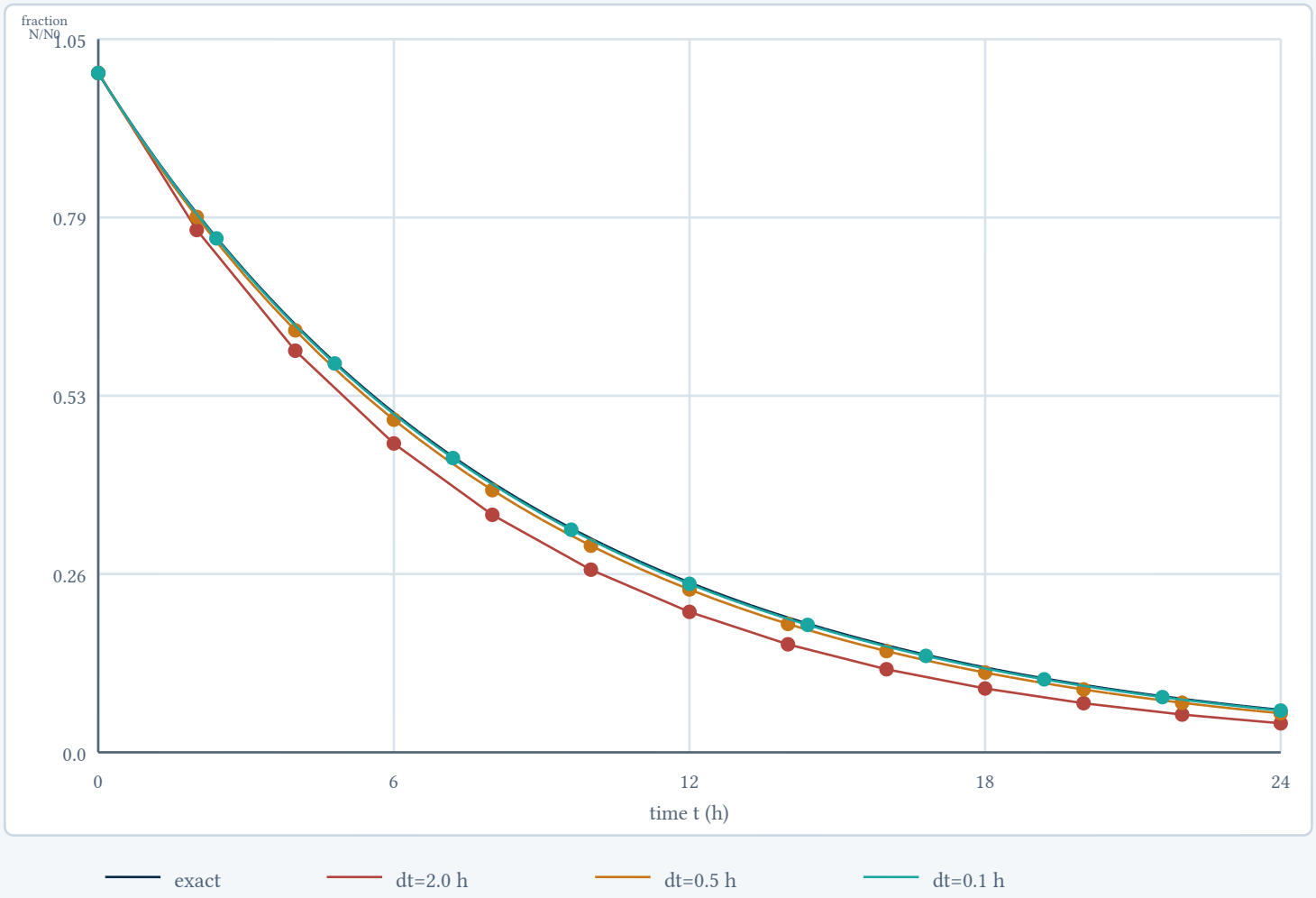
```
dtValues_h = [2.0 1.0 0.5 0.1];
for k = 1:numel(dtValues_h)
    % create t, run Euler, record result
end
```

Cost is part of the result

```
Steps for dt = 2.0, 1.0, 0.5, 0.1
h:
12, 24, 48, 240
```

The curves separate when dt is coarse

All four runs solve the same model; only the time resolution changes.



Read the shape

Coarse Euler steps fall below the exact curve at 24 h. Fine steps track it more closely.

Do not stop at “looks close”

The final value and an error measure make the comparison inspectable.

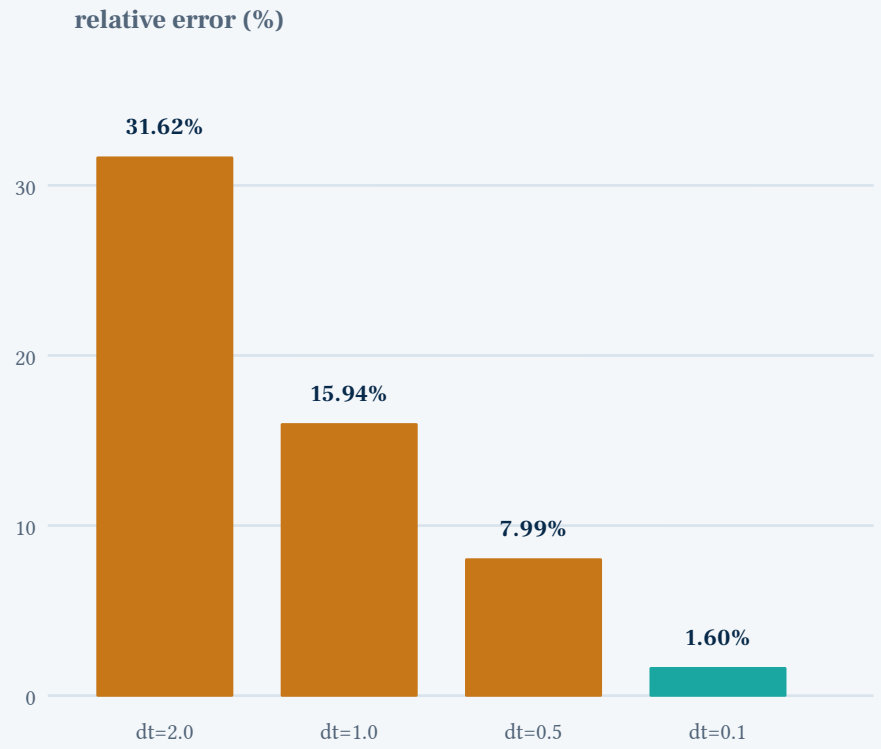
Question

Which run would you choose for a first report — and what evidence would you cite?

Smaller dt improves the result here — at a measurable cost

The table is the evidence behind the visual claim.

dt (h)	lambda*dt	Euler/N0 at 24 h	error	steps
2.0	0.2310	0.042735	31.62%	12
1.0	0.1155	0.052536	15.94%	24
0.5	0.0578	0.057505	7.99%	48
0.1	0.0116	0.061499	1.60%	240



A numerical result includes accuracy and cost.

Reference at 24 h

Exact N/N0 = 0.0625. The finest run is closest, but every claim should name the chosen dt and its error.

A trustworthy result leaves a trail of checks

Fresh run, visible assumptions, and a reference make the computation reproducible.

1. **Physics** — Question, variables, assumptions, expected trend
2. **Units** — lambda and t use the same time unit
3. **Algorithm** — Update rule, initial condition, index range
4. **Numerics** — At least two dt values and a relative error
5. **Communication** — Labels, units, legend, and one interpretation

VALIDATION SNIPPETS

```
assert(abs(N(1)-N0) < eps(N0))
assert(abs(N_exact_fraction(end)
    - 0.0625) < 1e-12)
assert(all(isfinite(N)))
assert(all(N >= 0))
```

Reproducibility record

Parameters + units
dt + tMax + step count
MATLAB version if relevant
validation evidence

VALIDATION CHECKS PASSED

AI can suggest syntax; it cannot certify physics

Keep the model, tests, and interpretation under human control.

A safe assistance pattern

1 Ask for an explanation or syntax suggestion
2 Read the code line by line
3 Check units and limiting cases
4 Compare with N_exact
5 State the test you performed

What the assistant cannot establish

Whether the physical assumptions fit the sample
Whether lambda and dt use compatible units
Whether indexing preserved the history
Whether the chosen accuracy is adequate for the question

Exit ticket / three sentences before you leave

1 **Model**
Equation + one assumption.

2 **MATLAB**
The line that advances one step.

3 **Evidence**
Why your chosen dt is acceptable.

CHECKPOINT / CAN YOU DEFEND YOUR DT?

Leave with a model, a method, and a test you can rerun

The Live Script carries today's experiment into the practical.

Takeaway 1

A computational result begins with a model and assumptions — not with a MATLAB command.

Takeaway 2

dt is part of the numerical method. Smaller is not automatically free.

Takeaway 3

Validation and interpretation are required parts of a simulation.

Next in the approved Week 1 package

1. **Run the Live Script**
   Keep the parameter record visible and start from a fresh session.
2. **Change one value at a time**
   Try T_half_h = 3.0, dt_h = 0.1, and N0 = 5000.
3. **Submit the evidence**
   One labelled plot, one limiting-case check, one interpretation.

Bring to the practical

A script or Live Script that can simulate decay for a chosen dt. Be ready to show:

- the update rule
- the timestep comparison
- why your chosen result is adequate

MODEL → CODE → EVIDENCE